

AI Engineering: 60-Day Learning Plan

12 weeks · 5 study days per week · 6 hours per day

For Sagar: build on your Python, SQL, data engineering, Airflow, and Docker experience. Focus on LLM applications, retrieval, agents, evaluation, and production practices. Check your math and ML knowledge through exercises; data engineering experience does not establish those prerequisites.

Schedule	Commitment
Study days	60, numbered D01–D60
Weekly rhythm	Monday–Friday; Saturday and Sunday off
Weekly hours	30
Total hours	360
Calendar span	12 weeks, about three months
Start date	Your choice; D01 is the first Monday you study

60 days means study days, not consecutive calendar days. You can choose another five-day pattern while keeping two days off. No compulsory weekend homework.

What changes from the old plan

Your detailed 45-day PDF allocated 8.5 hours per day: 382.5 hours. This plan allocates 360 hours, with shorter days and weekends off. It redistributes the work rather than adding 15 full days of content. You will study all three sources together, cover all 12 self-track project topics, and reuse working components instead of rebuilding databases, APIs, and dashboards for each project.

Scope: finish runnable learning builds, experiments, and one integrated portfolio capstone. The larger IMPLEMENTATION_PLAN.md files remain reference specifications. This schedule does not promise completion of every feature in those specs, eight separate polished Ed Donner products, or the entire From-Scratch curriculum. Use the completion criteria below for this plan; mark an original project complete in ROADMAP.md only after meeting its original requirements.

1. Your three sources

Source	Role in this plan	How to use it
Ed Donner: AI Engineer Core Track / LLM Engineering	Guided examples, open models, fine-tuning, and agents	Watch the topic named in the daily row; run the matching notebook, then modify it. Official course repository · Course resources
Self track: this folder	Project requirements and portfolio work	Use ROADMAP.md and the project specifications linked in Section 5. Build one reusable workspace with separate runnable project modules.
AI Engineering from Scratch	Algorithms and system mechanics	Read the selected lesson, implement the small exercise, and explain its connection to your project. Curriculum · direct lesson links in Section 6.

Notation: Ed W1 means Week 1 of Ed's course, not Week 1 of this schedule. FS P7-L02 means From-Scratch Phase 7, Lesson 02. R-P1 means self-track Project 1.

Ed's lecture numbers can change. Follow the **week + topic + notebook**, not the lecture ranges in the old PDFs. The public course repository confirms an eight-module course. This plan uses the topic mapping already in EXTENSIVE_PLAN.md: W1 fundamentals, W2 tools/UI, W3 open models/audio, W4 evaluation/code generation, W5 RAG, W6 datasets/SFT, W7 QLoRA, W8 agents. Study W5 before W4 here so you can evaluate a working retrieval system. Your purchased course's current section titles take precedence.

The live From-Scratch curriculum has changed since your old mapping. Use the lesson titles and links below. Ignore the old plan's phase-wide skip lists and its assumption that [CORE]/[DEEP] are current site labels.

2. A six-hour day

Block	Time	Activity	Total over 60 days
Learn	1.5 h	Ed video + notebook inspection, or one targeted FS lesson	90 h
Build	3 h	Implement the daily slice, including from-scratch exercises	180 h
Prove	1 h	Run the scenario, measure results, inspect failures	60 h
Wrap / catch-up	0.5 h	Fix a blocker, record evidence, recall concepts, save progress	30 h
Total	6 h	Breaks sit outside these study hours	360 h

Example clock: 09:00–10:30 learn; 10:45–12:15 build; 13:00–14:30 build; 14:45–15:45 prove; 15:45–16:15 wrap. Lunch and breaks make this a longer elapsed day, not extra study time.

- On an Ed + FS day, spend about 45 minutes on each within the 90-minute learning block. Save coding for the build block.
- Use 1–1.5× video playback only while you can follow the code. Stop to inspect inputs and outputs.
- Friday's build block is a **finish-and-repair block**, not three hours of extra features. The named checkpoint is your target.
- If you finish early, use the remaining time for a no-notes explanation or a failed scenario. Do not add another framework.
- If a checkpoint fails, use wrap blocks and the next Friday's repair block. Keep the six-hour ceiling and weekends off. Do not mark a dependency complete; if repair exceeds those blocks, shift dependent work and extend the finish date rather than claim completion.

Workspace and spending rules

Keep one workspace with labs/, rag/, agents/, document_pipeline/, evaluation/, gateway/, and capstone/. These are suggested folders for your future builds, not folders created by this document. Keep one shared dataset and a Docker Compose stack; enable Redis, Neo4j, and Airflow only when needed. Each project module needs its own run instructions and evidence.

Use synthetic or public documents. Start with one accessible small model and one embedding model; record model IDs, dimensions, versions, and pricing dates. Model names in old files are examples, not current recommendations. Do not compare unrelated embedding spaces, even if their dimensions match.

Set a spending cap you can afford before the first paid call; check usage daily. Use local models for iteration where practical. Book a short GPU session for Week 8 only after checking model access, memory needs, and cost. If a GPU is unavailable, run a small CPU LoRA exercise and mark QLoRA as studied, not executed. A paid SFT run is optional; preparing its data and evaluation is required. Never claim a run you did not perform.

3. Twelve-week overview

Week	Days	Focus	Friday checkpoint
1	01–05	LLM APIs, tokens, prompts, tool calls	Summarizer + tool-using assistant
2	06–10	Attention, GPT mechanics, Hugging Face, multimodal	Runnable attention lab + model/audio comparison
3	11–15	RAG baseline, corpus, chunking, embeddings	Cited RAG API + frozen evaluation questions
4	16–20	Hybrid retrieval, reranking, query transformations	R-P1 results + R-P2 security setup
5	21–25	Secure RAG, graph/agentive/multimodal retrieval	R-P2 security evidence + R-P3 comparison
6	26–30	Tool-using SQL agent and task evaluation	R-P4 safe multi-turn agent
7	31–35	Document processing, validation, neural-network basics	R-P5 pipeline + review queue
8	36–40	SFT/LoRA/QLoRA, Ed's agent capstone, orchestration	Fine-tuning report + R-P6 working graph
9	41–45	Multi-agent reliability and evaluation harness	R-P6 comparison + R-P7 scored report
10	46–50	Eval regression, prompt experiments, serving gateway	R-P7/R-P8 evidence + R-P9 routing baseline
11	51–55	Gateway reliability, privacy, capstone assembly	R-P9/R-P10 evidence + first full ticket flow
12	56–60	Capstone integration, deployment, case study	R-P11 reproducible demo + R-P12 case study

4. Day-by-day plan

Every row allocates **1.5 h learn + 3 h build + 1 h prove**. Add the **0.5 h wrap/catch-up block** from Section 2 to every day. Tick a day only after running its proof, not after watching its videos.

Week 1: LLM application fundamentals

Goal: call a model, understand its inputs, and give it a bounded tool. Choose a small public FAQ/document set you will reuse for RAG.

Day	Learn — 1.5 h	Build — 3 h	Prove / daily output — 1 h
[] D01 Mon	Ed W1: environment, Ollama, API calls; FS P0-L04	Set up dependencies and secrets outside Git. Run one local or hosted model. Capture usage, model ID, and latency.	Run three prompts from a clean terminal. Explain messages, tokens, and context window. Record your chosen spending cap.
[] D02 Tue	Ed W1: website summarizer, model comparison; FS P10-L01	Build a summarizer over saved public page text. Count tokens and compare two prompts on the same input.	Check five source facts against each summary. Save inputs, outputs, token counts, and unsupported claims.
[] D03 Wed	Ed W1: brochure generator; FS P11-L01	Adapt Ed's brochure example to your own public sample. Separate input loading, prompt, and output formatting.	Run on a second input without editing the code. Explain one prompt change and its observed effect.
[] D04 Thu	Ed W2: streaming UI and structured output; FS P11-L03	Build a streaming Gradio chat UI and a Pydantic-validated extraction call. Keep them as small labs.	Inspect streamed output; try missing fields and invalid types. Save validation outcomes, not just a successful screenshot.
[] D05 Fri	Ed W2: airline assistant and tool calling; FS P11-L09	Finish an assistant with a real local lookup tool and an explicit tool allowlist. Repair Week 1 blockers.	Demo normal lookup, unknown record, and invalid arguments. No real booking or purchase. Explain the model/tool execution boundary.

Friday gate: you can rerun the summarizer and assistant; no secrets appear in code; you can explain token cost and schema validation without notes.

Week 2: Model mechanics and open models

Goal: implement the important small algorithms. Do not train a full transformer from scratch this week.

Day	Learn — 1.5 h	Build — 3 h	Prove / daily output — 1 h
[] D06 Mon	FS P7-L02: self-attention; refresh matrix multiplication if needed	Implement scaled dot-product attention in NumPy or PyTorch using tiny tensors. Print Q/K/V and intermediate shapes.	Check attention rows sum to one; compare with a library reference on the same tensors. Explain the scale factor.
[] D07 Tue	FS P7-L03 and P7-L05: heads and transformer blocks	Extend your attention lab to multiple heads, residuals, and normalization. Sketch encoder and decoder differences.	Check tensor shapes and a reference output. Explain heads, residual connections, and cross-attention.
[] D08 Wed	FS P7-L07 and P10-L02: causal GPT and BPE	Implement causal masking and a tiny BPE merge loop. Inspect a pretrained tokenizer's encode/decode and chat template.	Show future-token masking and tokenizer round trips. Explain why your toy tokenizer is not a production tokenizer.
[] D09 Thu	Ed W3: Hugging Face pipelines/model inspection; FS P7-L12	Run one small open model. Inspect architecture, tokenizer, generation settings, and KV-cache behavior.	Save generation output and measured timing under fixed settings. Explain prefill/decode and KV-cache memory.
[] D10 Fri	Ed W3: audio/meeting-minutes example; FS P10-L11 overview	Run Ed's meeting-minutes workflow on a short public or synthetic audio clip. Finish the attention and tokenizer labs.	Check transcript and summary against the clip. Produce a one-page open-model/quantization tradeoff note; label explanations versus measured results.

Friday gate: attention and BPE labs run; you can draw a decoder block; one open-model and one audio workflow have saved evidence.

Week 3: Build and measure a RAG baseline

Goal: start R-P1 with a working baseline before optimization. Run Ed W5 here ahead of W4.

Day	Learn — 1.5 h	Build — 3 h	Prove / daily output — 1 h
[] D11 Mon	Ed W5: RAG pipeline/vector stores; FS P11-L06	Read R-P1's spec. Set up Postgres + pgvector; load 10-20 public Markdown/text/PDF documents with source metadata.	Recreate the database and inspect a retrieved source. Document corpus origin and ingestion failures.
[] D12 Tue	Ed W5: splitting; FS P5-L23	Implement fixed-size, semantic, and sentence-window chunking as selectable strategies. Preserve document and page/span references.	Inspect chunk boundaries on five documents. Save examples of lost context and overlapping duplicates.
[] D13 Wed	Ed W5: embeddings; FS P11-L04 and P5-L22 overview	Batch embeddings and store model/dimension metadata. Implement cosine similarity on a tiny sample before pgvector search.	Compare your cosine results to a reference. Verify query/document embeddings share the same model and space.
[] D14 Thu	Ed W5: knowledge worker, history, UI; FS P11-L05	Build retrieval → context → answer with source citations, an abstain path, FastAPI /query, and a small UI.	Ask supported, unsupported, and follow-up questions. Inspect retrieved chunks and cited claims for each.
[] D15 Fri	Ed W5: RAG evaluation; FS P10-L10	Finish the baseline. Create 30 labelled questions: 20 development and 10 held-out, including unanswerable cases.	Freeze splits and version the data. Save baseline retrieval scores, citation checks, latency, and cost. Do not tune on held-out cases.

Friday gate: a cited RAG answer works from the API; an unsupported question gets an abstention; the versioned corpus and baseline report support later comparisons.

Week 4: Retrieval engineering and security setup

Goal: finish R-P1's learning build and start R-P2 using the same retrieval components.

Day	Learn — 1.5 h	Build — 3 h	Prove / daily output — 1 h
[] D16 Mon	FS P5-L14: information retrieval; Ed W5 advanced retrieval	Implement a small true BM25 scorer, then dense + lexical retrieval with reciprocal rank fusion. Keep ranking inputs visible.	Compare lexical-only, dense-only, and fused rankings on development questions. PostgreSQL ts_rank_cd is lexical ranking, not BM25; label it correctly if used.
[] D17 Tue	FS P11-L07: reranking	Add a cross-encoder reranker after candidate retrieval. Compare top-k settings with the same data and model.	Record MRR/nDCG changes and reranking latency. Keep an improvement only if evidence justifies its cost.
[] D18 Wed	Ed W5: query rewriting/expansion; review FS P11-L07	Implement HyDE and multi-query as optional strategies. Deduplicate candidates and count added model calls.	Compare no transform, HyDE, and multi-query on development cases. Save failures and cost; no required improvement percentage.
[] D19 Thu	R-P1 evaluation/dashboard sections	Finish selectable retrieval configurations, a small results dashboard, and run instructions. Evaluate the chosen config on held-out cases once.	Reproduce baseline-versus-final report. Record retrieval quality, groundedness sample, latency, and cost. R-P1 learning build complete.
[] D20 Fri	R-P2: identity, roles, RLS; FS P11-L12 overview	Add two synthetic tenants, server-verified identity, document roles, and tenant-bound query context. Repair R-P1 blockers.	Prove unauthenticated rejection and tenant context derivation. Write the isolation test matrix for Week 5. Do not accept tenant identity from the question text.

Friday gate: R-P1 has runnable comparison evidence. R-P2 has authenticated tenant context and a written access-control test matrix.

Week 5: Secure and alternative RAG architectures

Goal: finish R-P2 and build R-P3 as a bounded architecture comparison on the existing corpus.

Day	Learn — 1.5 h	Build — 3 h	Prove / daily output — 1 h
[] D21 Mon	R-P2: retrieval filters, role checks, RLS	Enforce tenant/role filtering before retrieval and on source access. Configure RLS for the application DB role, which must not bypass RLS.	Try cross-tenant query, direct document access, forged tenant ID, and wrong role. Each forbidden request must fail without exposing content.
[] D22 Tue	R-P2: citations, audit, upload permissions	Add upload authorization, cited answers, tenant-scoped audit events, and append-only restrictions for the application role.	Run the whole isolation matrix; try audit UPDATE/DELETE as the app role. Save evidence and a README. R-P2 learning build complete.
[] D23 Wed	R-P3 GraphRAG; FS P5-L26	Extract entities/relations from five documents, preserve source evidence, store them in Neo4j, and answer a multi-hop question.	Compare graph and vanilla retrieval on five relationship questions. Inspect extraction mistakes and unsupported edges.
[] D24 Thu	R-P3 agentic retrieval; FS P14-L01	Add a bounded retrieve/check/retrieve loop with a maximum of three retrieval rounds and an abstain exit. Reuse tools from the baseline.	Run sufficient-context, insufficient-context, and repeated-query cases. Record termination, calls, cost, and quality versus vanilla RAG.
[] D25 Fri	R-P3 multimodal; FS P4-L18; survey KAG/LightRAG from the spec	Build a small CLIP text↔image retrieval lab. Add vanilla/graph/agentic/multimodal modes to a comparison entry point. Finish and repair.	Compare CLIP text to CLIP images in the same space, never BGE to CLIP. Save a mode-selection report with actual results. R-P3 learning build complete.

Friday gate: isolation tests pass; R-P3's modes run on bounded examples. Your report states when extra retrieval complexity helps and when it does not.

Week 6: Tool-using data analysis agent

Goal: build R-P4 with database permissions as the primary safety boundary. Learn Ed W4's evaluation and code-generation material alongside it.

Day	Learn — 1.5 h	Build — 3 h	Prove / daily output — 1 h
[] D26 Mon	Ed W4: task-specific model evaluation; FS P11-L09	Read R-P4's spec. Load Northwind or a synthetic sales DB. Create a restricted read-only role and schema/SQL tools.	Verify the role cannot mutate tables. Make a real schema-tool call and inspect its response.
[] D27 Tue	R-P4: SQL validation; FS P14-L06	Parse SQL with sqlglot, restrict schemas/statements, enforce row/time limits, and execute through the restricted role.	Try DML, DDL, multiple statements, unsafe functions, data-modifying CTEs, and an expensive query. Record rejected and timed-out cases.
[] D28 Wed	FS P14-L01 and P14-L02	Build a bounded tool loop, SQL error feedback, and at most two correction attempts. Return result provenance.	Answer five business questions; compare aggregates with hand-written SQL. Prove termination on an unsolvable question.
[] D29 Thu	Ed W4: Python-to-C++ example; FS P11-L05 review	Run Ed's code-conversion example on a small function with expected outputs. Add conversation state and one chart tool to R-P4.	Compare original/generated outputs before timing them. Exercise a follow-up filter and chart; state session/user isolation behavior.
[] D30 Fri	Ed W4: quality/cost/latency model selection	Finish the agent API/UI and run instructions. Compare two available models or two prompting strategies on the same tasks. Repair blockers.	Demo correct query, forbidden request, SQL correction, and follow-up. Save accuracy, calls, latency, and cost. R-P4 learning build complete.

Friday gate: the agent answers checked SQL questions, cannot mutate the DB, and stops after its allowed attempts. You have a task-specific model-selection note.

Week 7: Document processing and training prerequisites

Goal: build R-P5 by extending your ingestion work. Keep the document corpus small enough to inspect by hand.

Day	Learn — 1.5 h	Build — 3 h	Prove / daily output — 1 h
[] D31 Mon	Ed W6: data curation, splits, baselines; FS P3-L05	Read R-P5's spec. Prepare 20 synthetic mixed documents and expected fields. Add TXT/PDF/DOCX/EML loaders and OCR for one image sample.	Inspect extracted text against originals. Record parser/OCR failures, source IDs, and duplicate handling.
[] D32 Tue	Ed W6: neural networks; FS P3-L03	Implement a tiny forward/backward-pass lab and compare gradients with autodiff. Build a document classifier with structured output.	Check numerical gradients on the tiny lab. Score document labels against your hand-labelled sample; do not trust model self-confidence alone.
[] D33 Wed	Ed W6: SFT dataset format; FS P11-L03 review	Add Pydantic schemas for invoice/contract/email/receipt and extraction. Add arithmetic/date validators and bounded retry on errors.	Try missing fields, incorrect totals, and invalid dates. Route failures to review rather than silently accepting repaired guesses.
[] D34 Thu	R-P5: routing, Airflow, review queue	Build an idempotent Airflow DAG: ingest → classify → extract → validate → route. Store source IDs and add a Streamlit approve/edit/reject queue.	Run the batch twice without duplicating accepted records. Verify edits and rejections leave an audit trail.
[] D35 Fri	Ed W6: price-prediction example and ML/LLM comparison	Run a small version of Ed's price-prediction baseline. Finish the document API/UI, repair failures, and prepare a small fine-tuning dataset.	Process 20 documents end to end. Report extraction correctness, review rate, runtime, and cost; keep training/eval data separate. R-P5 learning build complete.

Friday gate: your pipeline validates records, routes uncertain data to review, and handles reruns. You can explain cross-entropy, gradients, and train/validation/test separation.

Week 8: Fine-tuning and orchestration

Goal: execute a small adaptation experiment and start R-P6. Ed's deal-spotting project is a guided lab, not a second portfolio capstone.

Day	Learn — 1.5 h	Build — 3 h	Prove / daily output — 1 h
[] D36 Mon	Ed W6 SFT recap; FS P10-L06/07/08 overview	Prepare a small SFT dataset and a held-out evaluation. Measure a prompted baseline. Inspect paid SFT setup; submit only within your chosen budget.	Validate dataset format and split independence. Write when prompt/RAG/SFT is appropriate. Record SFT execution status; RLHF/DPO are conceptual study here.
[] D37 Tue	Ed W7: LoRA/QLoRA; FS P11-L08	Implement a tiny LoRA layer, freeze base weights, and inspect trainable parameters. Prepare the course's small-model training notebook.	Verify frozen weights stay unchanged and adapters receive gradients. Check GPU/model access and estimate memory before a real QLoRA run.
[] D38 Wed	Ed W7: training, validation, checkpoints	Run a small QLoRA job on an accessible GPU, or a real small CPU LoRA training job if GPU access fails. Save config, adapter, loss, and predictions.	Compare base/prompted/adapted outputs on held-out data. Report task metric, training cost, and limitations. Do not infer improvement from training loss.
[] D39 Thu	Ed W8: deal-spotting agents and Modal deployment; FS P14-L12	Run Ed's agent workflow on saved sample deals, with real local retrieval and no purchases/messages. Read R-P6's spec; map router/research/writer/critic state.	Trace tool calls and completion in the guided lab. Explain Modal's deployment pattern; your portfolio deployment will use Docker.
[] D40 Fri	Ed W8: agent capstone; FS P14-L13	Implement a LangGraph router → research → writer → critic workflow, using real local knowledge lookup. Bound revisions at two and add a human-review exit.	Run one task through the graph and inspect state transitions. Repair fine-tuning/graph blockers; save a training report and an agent trace.

Friday gate: one adaptation experiment actually ran, with its method labelled. R-P6's graph runs and stops; no lookup stubs count as completion.

Week 9: Multi-agent reliability and evaluation

Goal: finish R-P6, then build R-P7 around the RAG and agent systems you already have.

Day	Learn — 1.5 h	Build — 3 h	Prove / daily output — 1 h
[] D41 Mon	FS P14-L25/28: collaboration and orchestration	Add a bounded orchestrator-worker example with concurrent independent subtasks and synthesis. Add persisted session state and per-node call/cost accounting.	Compare with a single-agent baseline. Simulate a worker failure; inspect failed/partial status and confirm unrelated session data stays isolated.
[] D42 Tue	FS P14-L26: agent failure modes	Add one checkpoint/resume path and a trace view. Finish R-P6 instructions and compare single-agent versus multi-agent on five tasks.	Prove revision limits, failed-tool behavior, resume correctness, and cost visibility. Recommend multi-agent only where results justify it. R-P6 learning build complete.
[] D43 Wed	FS P11-L10 and P5-L27; R-P7 specification	Build an eval runner with dataset/config version, model ID, case-level outputs, and RAG/agent adapters. Expand to 50 labelled cases without tuning on the held-out set.	Run a batch and inspect five cases by hand. Check that outputs preserve retrieved contexts, citations, failures, and usage.
[] D44 Thu	FS P10-L10 review: metrics and benchmark limitations	Implement retrieval recall/precision, MRR/nDCG, citation checks, task success, latency, and token-cost summaries.	Use known-correct and known-incorrect fixtures to verify metric math. Explain that citation validity alone does not establish claim support.
[] D45 Fri	FS P5-L27: faithfulness/relevance judges	Add structured judge rubrics and human labels for at least 20 outputs. Review judge disagreement and finish a report/dashboard. Repair blockers.	Save case-level scores and human/judge agreement, plus three failure analyses. Calibrate the rubric before using thresholds. Never require a fabricated target score.

Friday gate: R-P6 has reliability evidence and a single-agent comparison. R-P7 produces a reproducible report with human-checked examples.

Week 10: Regression, prompt experiments, and gateway

Goal: finish R-P7/R-P8 and begin R-P9. Use one evaluation harness rather than three separate dashboards.

Day	Learn — 1.5 h	Build — 3 h	Prove / daily output — 1 h
[] D46 Mon	FS P11-L05 and P11-L10; R-P7 context/CI sections	Evaluate context length/order and truncation. Add regression checks against a versioned baseline, plus a scheduled/manual live-eval path.	Deliberately change a retrieval/prompt config and observe a detected regression. Reproduce the report. R-P7 learning build complete.
[] D47 Tue	R-P8: versioning, templates, promotion/rollback	Store immutable prompt versions, render with strict variables, and link each eval run to a version. Add explicit promotion and rollback.	Render two versions on identical inputs; try missing variables. Verify promotion/rollback selects the recorded version.
[] D48 Wed	R-P8 statistical comparison; refresh paired data analysis	Run an offline paired A/B experiment on the same cases. Report paired score differences, uncertainty, failures, cost, and latency.	Use a bootstrap paired confidence interval or McNemar for paired binary outcomes. Small samples may be inconclusive. Finish R-P8 learning build.
[] D49 Thu	FS P11-L11; R-P9: serving gateway	Put model calls behind a FastAPI gateway. Add caller/tenant context, configured model routing, request IDs, and current token-price records.	Send real requests through two routes. Check usage math against provider-reported tokens. Do not use LLM self-confidence as a calibrated routing signal.
[] D50 Fri	R-P9: cache scope; review FS P11-L11	Add exact caching keyed by tenant, role/access scope, prompt/model/config, and corpus version. Add a bounded semantic-cache experiment for eligible public queries. Repair blockers.	Show eligible cache hit, different-tenant miss, corpus-version invalidation, and a similar-but-different question. Explain why semantic caching can return a wrong answer.

Friday gate: R-P7 regressions run; R-P8 has an honest experiment outcome; the gateway handles routed calls and safe cache separation.

Week 11: Reliability, privacy, and first capstone flow

Goal: finish R-P9/R-P10 before moving their components into R-P11.

Day	Learn — 1.5 h	Build — 3 h	Prove / daily output — 1 h
[] D51 Mon	R-P9: rate limits, budgets, provider failover	Add tenant rate/budget limits, bounded timeouts/retries, and a configured alternate provider or local model. Add a cost/latency/cache report.	Exercise burst requests, exhausted budget, and injected provider outage. Run one real alternate-model request. Report measured costs, not the old plan's promised savings. R-P9 learning build complete.
[] D52 Tue	R-P10: PII detection; FS P5-L06	Add regex + Presidio/spaCy detection for synthetic English/German names, email, phone, IBAN, and tax-ID examples. Implement masking and tenant allowlists.	Score a hand-labelled sample with false positives and negatives. Inspect overlap handling; a postcode alone is not proof of sensitive identity.
[] D53 Wed	FS P11-L12 and P14-L27; R-P10 audit/redaction sections	Scan inputs and outputs; keep raw PII out of logs and hosted detector calls. Add a local contextual-detector experiment, audit events, and prompt-injection cases.	Trace synthetic PII through the proxy. Verify masks and retained audit metadata; test malicious retrieved instructions. Document residual risk. R-P10 learning build complete, not a GDPR certification.
[] D54 Thu	R-P11: ticket state machine and component interfaces	Assemble ticket ingestion → redaction → classification → secure retrieval → draft. Reuse the working gateway, prompt store, and audit layer.	Process five synthetic tickets from two tenants. Inspect citations and trace IDs; make sure redaction occurs before hosted model calls.
[] D55 Fri	R-P11: human review and feedback	Add review/approve/edit/reject states and store evaluation feedback. Use retrieval/eval evidence for routing; never auto-send real customer messages. Repair integration blockers.	Show supported ticket, unsupported ticket, sensitive ticket, and human edit. Save a first end-to-end demo. Customer delivery remains a local simulated outbox.

Friday gate: gateway failures and privacy cases have evidence; the capstone processes tickets end to end with human review and tenant isolation.

Week 12: Capstone proof and technical case study

Goal: finish a reproducible portfolio system with measured results and honest limitations. Stop adding features.

Day	Learn — 1.5 h	Build — 3 h	Prove / daily output — 1 h
[] D56 Mon	R-P11: orchestration and operational behavior	Integrate the bounded agent workflow where ticket type justifies it. Add explicit failures/review states and safe rerun behavior for interrupted tickets.	Reprocess the same ticket without duplicate resolution. Check tenant isolation in retrieval, cache, conversation state, and audit access.
[] D57 Tue	R-P11 + R-P7: evaluation feedback loop	Run at least 30 synthetic tickets across supported, unsupported, adversarial, and sensitive cases. Use a held-out subset for the final report.	Report groundedness, citation support, task success, review rate, p50/p95 latency, and cost/ticket. Retain failed cases rather than excluding them.
[] D58 Wed	FS P11-L13; R-P11 deployment/runbook sections	Finish Docker Compose startup, health checks, sample data loading, and a dashboard covering queue, traces, evals, costs, and audits.	Start from a clean database and follow the README. Exercise provider outage and review recovery. Save screenshots plus command output. R-P11 learning build complete.
[] D59 Thu	R-P12: case study and architecture decisions	Write an 8–12-page case study using your actual results. Include architecture, data flow, five ADRs, failures, operational notes, privacy limits, and component-to-project mapping.	Trace every result to a saved run. Explain retrieval choice, agent complexity, model choice, cache boundaries, and review policy without notes.
[] D60 Fri	Review your three-source learning notes; no new course content	Repair final demo blockers. Finish a 10–15-slide deck, a 5–10-minute demo recording, and the workspace/project index.	Run the capstone from its instructions. Present one successful and one failed/reviewed ticket. Finish R-P12 and the completion checklist; label any unmet criterion.

Final gate: another engineer can run the capstone, inspect measured results, and understand your decisions. The plan is complete when the criteria below are met, not when day 60 arrives.

5. Self-track project coverage and completion criteria

Use the linked specs for design detail. These are **learning-build criteria for this 360-hour plan**, not replacements for the original specs' full acceptance criteria. Shared code is encouraged; each project still needs a runnable example, evidence, and a short explanation.

Project and original specification	Scheduled days	Evidence required by this plan
R-P1 Advanced RAG	11-19	Three chunkers; lexical/dense/fused rankings; reranker; HyDE/multi-query comparison; cited API/UI; baseline and held-out report.
R-P2 Secure RAG	20-22	Two tenants; auth/roles; pre-retrieval filtering and RLS; upload/source authorization; citations; app-role append-only audit; negative isolation tests.
R-P3 Advanced RAG Architectures	23-25	Runnable graph, bounded agentic, and CLIP multimodal examples; comparison entry point; KAG/LightRAG survey; architecture-choice report.
R-P4 Data Analysis Agent	26-30	Real schema/SQL/chart tools; restricted DB role; validation/limits; bounded correction; checked answers; session-scoped multi-turn state.
R-P5 Document Processing Pipeline	31-35	Mixed loaders + OCR sample; classification; typed extraction; business validation; bounded retry; idempotent DAG; review UI and audit trail.
R-P6 Multi-Agent Systems	39-42	Router/research/writer/critic graph; bounded revisions; real lookup; orchestrator-worker example; persistence/resume; per-node cost; single-agent comparison.
R-P7 Evaluation Harness	43-46	Versioned 50-case set; deterministic metrics; calibrated judge sample; context experiment; regression checks; case-level report/dashboard.
R-P8 Prompt Versioning and A/B Testing	47-48	Immutable versions; strict rendering; paired offline experiment with uncertainty; explicit promotion/rollback; no forced winner.
R-P9 LLM Serving Gateway	49-51	Routing; tenant-scoped cache/invalidation; semantic-cache experiment; rate/budget limits; failover proof; request/cost/latency reporting.
R-P10 PII Guardrails	52-53	Regex/NER/contextual comparison; input/output masks; allowlist behavior; synthetic English/German sample; no raw PII in hosted detector calls/logs; audit and injection evidence.

Project and original specification	Scheduled days	Evidence required by this plan
R-P11 Customer Support AI	54-58, 60	Integrated tenant-safe ticket flow; review/feedback; safe reruns; simulated outbox; 30-ticket report; outage scenario; clean startup and operational dashboard.
R-P12 Technical Case Study	59-60	Measured case study; five ADRs; failure analysis/runbook; 10-15-slide deck; demo recording and workspace index.

Ed Donner coverage

Course module	Study days	Expected lab evidence
W1: APIs, tokens, first product	01-03	Summarizer and brochure on your own saved public inputs
W2: UI, tool calling, multimodal introduction	04-05	Streaming UI, structured extraction, safe assistant lookup
W3: Hugging Face, open models, audio	09-10	Open-model inspection and meeting-minutes sample
W4: evaluation and code generation	26, 29-30	Task comparison and checked Python-to-C++ example
W5: RAG and knowledge worker	11-18	RAG baseline and retrieval comparisons; reuse self-track code
W6: datasets, baselines, SFT	31-36	Curated data, checked baseline, SFT preparation; paid run optional
W7: LoRA/QLoRA	37-38	LoRA layer and one actual adaptation run; label GPU/CPU method
W8: agents and deployment patterns	39-40	Saved-deals workflow, real tool trace, bounded graph; Docker portfolio deployment

6. Selected From-Scratch lesson links

These links identify the assigned lessons; they are not an instruction to complete entire phases. **Implement:** write a small working exercise in the build block. **Apply:** read the lesson and use the idea in the project. **Overview:** explain the tradeoff without claiming a training or deployment run.

ID	Lesson	Depth
P0-L04	APIs & Keys	Apply
P3-L03	Backpropagation from Scratch	Implement
P3-L05	Loss Functions	Apply
P4-L18	Open-Vocabulary Vision: CLIP	Apply
P5-L06	Named Entity Recognition	Apply
P5-L14	Information Retrieval & Search	Implement BM25/RRF
P5-L22	Embedding Models Deep Dive	Overview
P5-L23	Chunking Strategies for RAG	Apply
P5-L26	Relation Extraction & Knowledge Graph Construction	Apply
P5-L27	LLM Evaluation: RAGAS, DeepEval, G-Eval	Apply
P7-L02	Self-Attention from Scratch	Implement
P7-L03	Multi-Head Attention	Implement
P7-L05	The Full Transformer: Encoder + Decoder	Apply; small block only
P7-L07	GPT: Causal Language Modeling	Implement causal mask
P7-L12	KV Cache, Flash Attention & Inference Optimization	Overview + KV-cache experiment
P10-L01	Tokenizers: BPE, WordPiece, SentencePiece	Apply

ID	Lesson	Depth
P10-L02	Building a Tokenizer from Scratch	Implement toy BPE
P10-L06	Instruction Tuning: SFT	Apply
P10-L07	RLHF: Reward Model + PPO	Overview
P10-L08	DPO: Direct Preference Optimization	Overview
P10-L10	Evaluation: Benchmarks, Evals	Apply
P10-L11	Quantization: INT8, GPTQ, AWQ, GGUF	Overview
P11-L01	Prompt Engineering	Apply
P11-L03	Structured Outputs	Apply
P11-L04	Embeddings & Vector Representations	Implement cosine similarity
P11-L05	Context Engineering	Apply
P11-L06	RAG: Retrieval-Augmented Generation	Implement baseline
P11-L07	Advanced RAG: Chunking, Reranking	Apply
P11-L08	Fine-Tuning with LoRA & QLoRA	Implement small LoRA
P11-L09	Function Calling & Tool Use	Apply
P11-L10	Evaluation & Testing	Apply
P11-L11	Caching, Rate Limiting & Cost	Apply
P11-L12	Guardrails & Safety	Apply
P11-L13	Building a Production LLM App	Apply

ID	Lesson	Depth
P14-L01	The Agent Loop	Implement
P14-L02	ReWOO and Plan-and-Execute	Overview
P14-L06	Tool Use and Function Calling	Apply
P14-L12	Anthropic's Workflow Patterns	Apply
P14-L13	Stateful Graph Orchestration	Apply
P14-L25	Multi-Agent Debate and Collaboration	Overview
P14-L26	Failure Modes: Why Agents Break	Apply
P14-L27	Prompt Injection and the PVE Defense	Apply
P14-L28	Orchestration Patterns	Apply

Outside the 60-day plan: full pretraining, distributed training, exhaustive computer vision/audio/diffusion/RL phases, every agent framework, and a second cloud deployment stack. Revisit these for a specific project or role. Do not skip math or ML topics you fail to explain during the exercises.

7. Progress and completion

For the 30-minute wrap block, record: **what ran; evidence path; what failed; one concept explained from memory; tomorrow's first action.** Keep a weekly note rather than creating a separate polished report every day.

Week	Study days complete / 5	Friday gate passed?	Evidence location	Blocker / next action
1				
2				
3				
4				
5				
6				
7				
8				
9				
10				
11				
12				

Final completion checklist

- [] Runnable attention, causal-mask, BPE, cosine, BM25/RRF, backprop, and LoRA exercises with reference checks.
- [] Ed's eight modules covered at the topics/depth stated here; execution and study-only work labelled.
- [] All 12 self-track topics have the learning-build evidence in Section 5; original full-spec completion remains tracked separately.
- [] Versioned datasets, fixed held-out splits, saved configurations, model IDs, and actual quality/latency/cost results.
- [] Negative tests for tenant isolation, forbidden SQL, loop limits, cache boundaries, PII leakage, and provider outage.
- [] One capstone runs from a clean start with sample data, review queue, operational dashboard, and simulated delivery.
- [] Case study, five ADRs, deck, and demo use results you can trace to saved runs.
- [] You can explain one failed experiment and why you kept the simpler design.

Reference notes

Local basis: 45-Day_AI_Engineering_Study_Plan_Detailed.pdf, EXTENSIVE_PLAN.md, ROADMAP.md, and the project specifications. Public source check: Ed's official repository and the From-Scratch lesson index. Websites can change; the assignment titles and links identify what to study rather than guaranteeing their claims or benchmark numbers.

The original 45-day files remain unchanged. This is the current 60-study-day schedule; the older documents supply design detail and historical context.

8. Extra capstone: City Disruption Memory

Keep D01–D60 and the customer-support capstone above. The additional project is **City Disruption Memory**: an AI-operated data engineering platform using live Chicago transit alerts, 311 infrastructure reports, and versioned geography. The primary objective is to configure, deploy, diagnose, and operate Spark/Docker/Terraform infrastructure with AI assistance and Prometheus/Grafana monitoring.

Construct your own collection history and derived dataset rather than clone a tutorial. Connect source evidence to Spark runs, infrastructure/configuration versions, alerts, AI diagnoses, approved actions, and verified recovery. City-text extraction and explanation are secondary features. The project makes no “world-first” claim.

Proposed extension	Focus	Hours
Week 13, D61–D65	Live collection, Docker/Spark deployment, Terraform ownership, Prometheus/Grafana telemetry	30
Week 14, D66–D70	Replay-safe history, Spark tuning experiments, AI diagnosis, configuration proposals, approval gates	30
Week 15, D71–D75	Verified recovery, held-out incident evaluation, clean startup, runbooks, operating case study	30
Extra capstone total	15 study days, five days/week, six hours/day	90

The original course remains **60 study days / 360 hours**. Taking the extension makes the combined schedule **75 study days / 15 weeks / 450 hours**. It is not hidden inside the original six-hour days.

See the capstone specification for all 15 daily tasks, infrastructure ownership, source/licensing boundaries, operational evaluation, and completion gates. Start collectors on D62 and retain seven calendar days of observations; unattended weekend jobs do not require weekend study. Preserve historical knowledge cutoffs and label captured-data load replay and injected faults as experiments, not new city observations.

Start locally: Terraform manages a Docker network and persistent volumes; Compose manages service containers, including a Spark master and two workers. Cloud provisioning is a separate provider/cost/permission decision, not an implied achievement. The AI assistant starts read-only; infrastructure changes and consequential recovery require exact-scope approval.

The 90-hour extension is a target that assumes reuse from the core course, not a guaranteed finish date. The specification does not implement the platform. Complete the monitoring, diagnosis, incident, and data-correctness gates before claiming successful AIOps.